

Credit-Card Fraud Detection: A Critical Reproduction

Course: שימוש בשיטות של דאטה סיינס בסייבר (Data Science in Cyber, 203.3888), University of Haifa

Student: Abdallah Khaldi, ID 212389712

Date: 9 July 2026

Tutorial critiqued: <https://github.com/sergi-s/Credit-Card-fraud-detection>

Dataset: ULB “Credit Card Fraud Detection”, <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

Code for this project: <https://github.com/AbdallahKhaldi/creditcard-fraud-critique>

1. Summary of the Source

Credit-card fraud detection matters for two reasons that pull against each other. The stakes are large: card fraud is a multi-billion-dollar annual loss worldwide, the attackers actively adapt (unlike static noise, fraud is adversarial), and it is a cybersecurity problem because it is automated theft against financial systems at scale. But the signal is tiny: in the dataset used here only 492 of 284,807 transactions (0.173%) are fraud, so any detector operates at the edge of what a review team can afford to inspect, and naive metrics stop meaning anything.

The tutorial under review is a public GitHub project that trains fraud classifiers on the ULB dataset: 284,807 card transactions collected over two days (in September 2013, per the dataset’s Kaggle description), with features Time (seconds since the first transaction), Amount, and V1-V28, which are anonymized PCA components of undisclosed original features.

The author’s approach, read from the repository’s notebooks: preprocess the data, compare several class-imbalance strategies (plain, class-weighted, under-sampled, and SMOTE over-sampled), train a fully-connected Keras/TensorFlow neural network, and evaluate on a held-out test set, reporting the SMOTE model as the headline. The repository’s other notebooks are later variants of the same experiment, two of them named for removing leakage and one for understandability. The headline result, quoted from the README, is that the oversampled model can “detect 100% of all fraudulent transactions in the unseen test set”, with reported test-set scores of F1 0.998, recall 0.9998 and precision 0.997.

(The claim and the results tables quoted in this report were taken verbatim from the repository README as accessed on 7 July 2026.)

The rest of this report examines whether the evidence supports the claim.

2. Critical Evaluation

The author’s claims. Read plainly, the tutorial makes four:

1. **C1** — the SMOTE model detects 100% of fraud in the unseen test set (F1 0.998).
2. **C2** — SMOTE oversampling is the right fix for the class imbalance.
3. **C3** — the reported accuracy and ROC-AUC demonstrate a strong detector.

4. **C4** — that test set is genuinely “unseen”.

My verdict, argued below: **C1 and C4 are not supported** (the test set is contaminated, so the number is an artifact), **C3 is misleading** (accuracy and ROC-AUC are the wrong metrics at this prevalence), and **C2 is right in spirit but fatally misapplied** (SMOTE is a reasonable tool, but applied before the split it becomes the leak). The project’s thesis:

The reported near-perfect performance is not evidence of a strong fraud detector. It is an artifact of (1) resampling the data *before* the train/test split, which leaks synthetic minority points into the test set, and (2) reporting accuracy and ROC-AUC, which are uninformative at 0.17% prevalence. Under a leakage-free protocol and imbalance-aware metrics, realistic performance is materially lower.

The mechanism. SMOTE creates each synthetic fraud example by interpolating between a real fraud and one of its nearest fraud neighbours. Applied to the full dataset before splitting, it plants near-copies of training rows inside the test set, so the model is evaluated on data it has effectively already seen. A second effect compounds it: the resampled “test set” is roughly half fraud instead of 0.17%, and at 50/50 prevalence precision is high almost by construction. These are two distinct causes — near-copy memorization inflates recall, and the balanced test prevalence inflates precision — and both follow from the single decision to resample before splitting.

Reproducing the flaw. I reproduced the tutorial’s protocol (SMOTE on the full dataset, split afterwards) with plain scikit-learn models rather than the author’s neural network, precisely to show the inflation is a property of the protocol and not of any particular model. On the contaminated split, logistic regression reaches F1 0.979 and a histogram gradient-boosting model reaches F1 0.9995 with recall that displays as 1.0000. The notebook’s confusion matrix shows even that “100%” is rounding: 3 of 84,976 contaminated-test frauds slip through. The tutorial’s own results table has the same wrinkle, listing recall 0.9998 beneath a “100%” headline.

The tutorial refutes itself. The README reports the oversampled model at F1 0.998 on its balanced “test set” but F1 0.570 (precision 0.399) when the same model is scored on the original, imbalanced data. That collapse is the fingerprint of SMOTE-before-split, and it also isolates the two effects: precision is what craters (0.997 to 0.399) when the test prevalence returns to reality, exactly as the “balanced-test” argument predicts. The repository also contains notebooks named “Deep Learning + Sampling + no leakage” and “Nodataleakage”, added alongside the original, which indicates the author became aware of the issue; the README’s headline claim was not revised.

The metrics. Accuracy is uninformative here: predicting “legitimate” for every transaction scores 99.83%. ROC-AUC misleads under extreme imbalance, and my own results demonstrate it: the random forest and the isolation forest score nearly identical ROC-AUC (0.941 vs 0.940) while their PR-AUCs differ by a factor of six (0.807 vs 0.128). These failure modes of accuracy and ROC-AUC under leakage and imbalance are documented systematically in recent critical studies of this exact literature (arXiv:2506.02703; arXiv:2412.07437).

Verdict. The claim “detect 100% of all fraudulent transactions in the unseen test set” is not supported. The test set was not unseen in any meaningful sense. Under the honest protocol of Section 5, the same logistic regression falls from F1 0.979 to 0.097, and the best corrected model, a class-weighted random forest, reaches F1 0.830 / MCC 0.836 / PR-AUC 0.807.

For fairness: not every boastful README in this space hides a leak. Of the other repositories I screened, anshpandey96 advertises “100% recall, 94% precision” (numbers that likely come from its synthetic fallback demo data) and Sudev18 oversells a plain SMOTE-plus-ROC-AUC setup, yet both actually split before resampling. The difference between those repositories and the one critiqued here is only visible in the code, which is the real lesson.

3. Feature Engineering Analysis

What the tutorial did. It standard-scales Amount with a StandardScaler fitted on the full dataset before the split (a second, smaller leak), drops Time, and leaves the V-features untouched; there is no domain feature construction.

What I did, and whether it helped. A stratified 70/30 split first, then RobustScaler on Time and Amount only (median/IQR scaling, robust to the heavy Amount tail), fit on the training set and reused inside every model pipeline. V1-V28 were left untouched. To avoid asserting that this choice matters, I ran a small ablation (notebook Section 4) scoring both supervised models on the held-out set under RobustScaler, StandardScaler, and no scaling. The result is almost a null one, and that is the finding: the random forest is identical to three decimals across all three (it only cares about split order), and logistic regression is the same under RobustScaler and StandardScaler (F1 0.097 vs 0.095) while unscaled it fails to converge in 1000 iterations. So scaling is the right call for the linear model (convergence, not accuracy, is the reason), the *choice* of scaler barely matters, and either way it moves almost nothing — unsurprising when 28 of 30 features are already scaled PCA components.

Encoding. There are no categorical predictors, so no encoding was required; had one existed (say, merchant category), the course recipe would apply: one-hot for low cardinality, target encoding for high cardinality. I considered adding $\log_{1p}(\text{Amount})$ and declined: it is rank-identical to Amount (Spearman 1.000), so trees gain nothing, and Amount carries almost no marginal signal for the linear model anyway (Spearman vs Class: -0.008).

Redundancy, selection, dimensionality reduction. The redundancy check (notebook Sections 1, 2, 4) confirms there are no constant columns, no literally duplicated columns, and no near-duplicate V-features (max absolute Pearson 0.019 on the full data, exactly 0.000000 on the raw file — the small nonzero value appears only because I removed duplicate rows after the PCA was made). I deliberately do not perform feature selection or dimensionality reduction: with 30 orthogonal features over 283,000 rows there is no curse of dimensionality and no overfitting pressure to relieve, and re-applying PCA/UMAP would be circular because the inputs are already principal components.

The ceiling. The interesting finding is how little feature engineering is possible. The V-features are anonymized PCA components: no units, no meaning, no way to construct domain features from them, and no way to explain to an analyst why a transaction was flagged. There is not even a card identifier to group by. With access to raw data, the features worth building are transaction velocity (time since the card’s previous transaction), merchant category, and geographic distance from the previous transaction. The tutorial never mentions this ceiling; it presents a dataset where feature engineering is impossible as if nothing were missing.

4. Reproducibility Analysis

The tutorial. I deliberately did not execute the tutorial’s notebooks: they are untrusted third-party code, and the headline model is a Keras/TensorFlow network outside this course’s stack. Everything below comes from reading the repository as of 7 July 2026. It contains four notebooks with saved outputs committed (67 of the main notebook’s 142 cells carry outputs), so the published numbers can be inspected without running anything.

Hidden preprocessing steps? Nothing is concealed in obfuscated code, but two leaks hide in plain sight and neither is flagged in the README: `smote.fit_resample(X, y)` is called on the full dataset and only afterwards is the resampled data split with `train_test_split(X_resample, y_resample, test_size=0.3)` (no `random_state`, no stratification), and the `StandardScaler` on Amount is likewise fit on the full dataset before the split. So the pipeline both leaks and is not bit-for-bit reproducible from its own notebook.

Files and dependencies? The dataset ships inside the repository (`creditcard.csv.zip`, 68.7 MB), so the inputs are available, but there is no `requirements.txt` or environment file, dependency versions are unpinned, and a stray `.ipynb_checkpoints` directory is committed. Overall the artifacts needed to *verify the flaw* are all present (committed outputs, committed data, README tables), but the pipeline itself is only loosely reproducible. The decisive fact, the F1 0.998 vs 0.570 collapse, is reproducible directly from the author’s own published tables.

This project. The accompanying notebook runs top-to-bottom on a fresh kernel in a few minutes (about 4.5 on this laptop, most of it the cross-validation and the scaler ablation, which re-fit the models several times), with pinned dependencies (`requirements.txt`), a fixed seed (`random_state=42`) everywhere, and dataset-download instructions in `data/README.md` (the 144 MB CSV is deliberately not committed). A fresh top-to-bottom run reproduces the committed outputs exactly, number for number and figure for figure. Every number in this report traces to a visible notebook cell.

5. Experimental Results

Protocol. After removing 1,081 exact duplicate rows (284,807 → 283,726; 492 → 473 frauds), I made one stratified 70/30 split (test: 85,118 rows, 142 frauds, 0.167%) and held it fixed for every model. Preprocessing and resampling live inside model pipelines, so nothing is ever fit on test data. Four models ran under the correct protocol — two supervised (logistic regression, random forest) and two unsupervised anomaly baselines (isolation forest, Mahalanobis distance) — and two under the tutorial’s flawed protocol for comparison.

Model	Protocol	Precision	Recall	F1	F2	MCC	ROC-AUC	PR-AUC
LogReg + SMOTE (in-pipeline)	correct	0.052	0.880	0.097	0.209	0.209	0.966	0.689
Random forest (class-weighted)	correct	0.946	0.739	0.830	0.773	0.836	0.941	0.807
Isolation forest (novelty)	correct	0.204	0.275	0.234	0.257	0.235	0.940	0.128
Mahalanobis (unsupervised)	correct	0.021	0.845	0.042	0.097	0.128	0.951	0.417
LogReg	leaky	0.991	0.967	0.979	0.972	0.958	0.997	0.997
HistGradientBoosting	leaky	0.999	1.000	0.9995	1.000	0.999	1.000	1.000

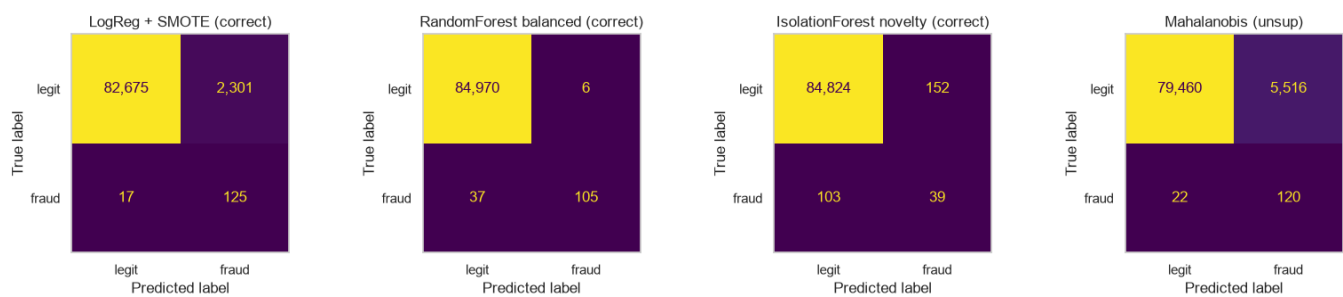
The leaky HistGB's recall, F2 and AUCs round to 1.000 at three decimals; the notebook's confusion matrix shows 3 of 84,976 contaminated-test frauds are still missed. I note this because rounding-to-perfect is part of what this report critiques.

Metric definitions and choices. The brief asks for the maths and the meaning of every metric, so (with TP/FP/TN/FN the confusion-matrix counts and positive = fraud):

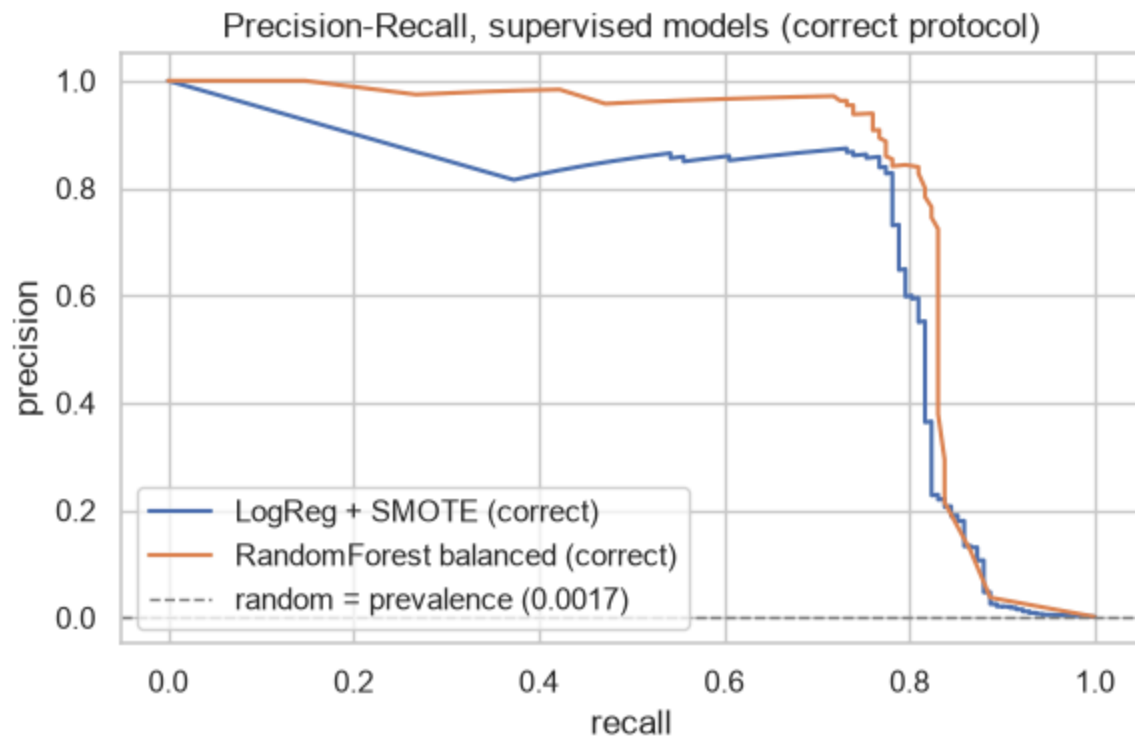
Metric	Formula	What it measures	Fraud meaning (FP/FN)
Precision	$TP / (TP + FP)$	of flagged, share truly fraud	low → analysts buried in false alarms
Recall	$TP / (TP + FN)$	of frauds, share caught	a miss → fraud succeeds, money lost
F1	$2PR / (P + R)$	harmonic mean of the two	balances misses vs false alarms
F2 (Fβ, β=2)	$(1+\beta^2)PR / (\beta^2P + R)$	recall weighted $\beta^2=4\times$ precision	missed fraud costs more than a false alarm
MCC	$(TP \cdot TN - FP \cdot FN) / \sqrt{((TP+FP)(TP+FN)(TN+FP)(TN+FN))}$	prediction–truth correlation in [−1,1]	honest under 600:1 imbalance
Accuracy	$(TP + TN) / N$	overall fraction correct	useless: 99.83% by always saying “legit”
ROC-AUC	P(score of a random fraud > score of a random legit)	ranking quality across thresholds	over-optimistic under imbalance
PR-AUC	area under the precision–recall curve	precision-weighted ranking	the honest headline at 0.17% prevalence

I exclude accuracy as a headline (the base rate makes it meaningless) and keep but de-emphasize ROC-AUC (it barely moves when 85,000 legit rows dominate). I *add* PR-AUC, which is not on the course's standard list, because at 0.17% prevalence it is the metric that actually separates the models. F1 is used only for the leaky-vs-honest comparison, because it is the tutorial's own headline metric, so the collapse is measured on their chosen ground; F2, MCC and PR-AUC are the operational headline.

The cleanest comparison is logistic regression against itself: F1 0.979 under the leaky protocol, 0.097 under the honest one. It is the same algorithm; the only change is where SMOTE runs (which in turn changes the test-set prevalence), and 0.88 of F1 evaporates. The honest random forest, the best model here, reaches F1 0.830 / MCC 0.836 / PR-AUC 0.807 (105 of 142 frauds caught, 6 false alarms in 85,118 transactions), in the range leakage-aware studies report for this dataset (arXiv:2506.02703; arXiv:2412.07437).



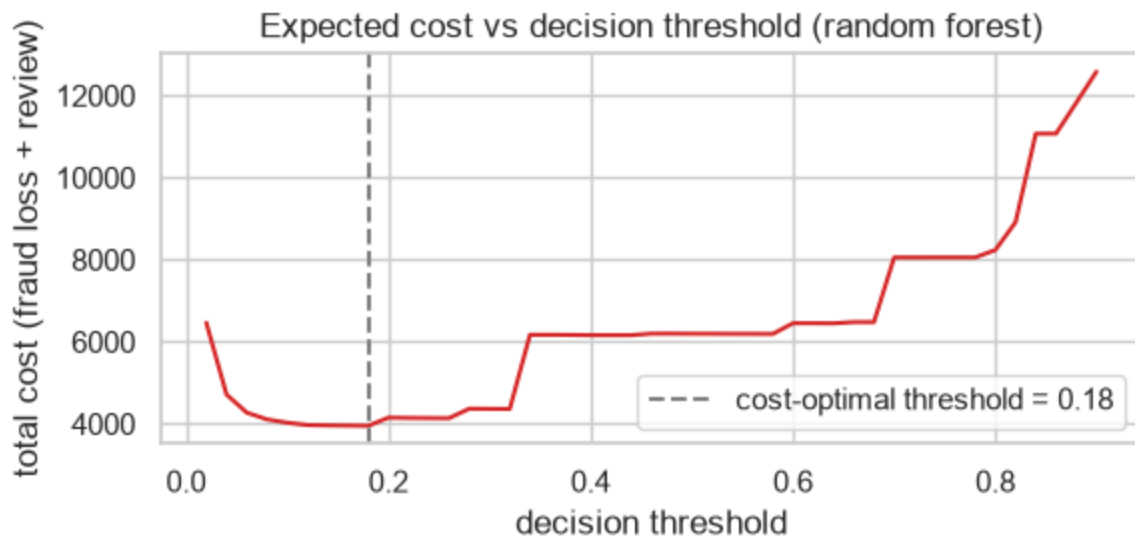
Confusion matrices of the four correct-protocol models on the held-out test set (from the notebook, Section 6). The random forest's 6 false alarms versus the anomaly models' hundreds-to-thousands is the operational story.



Precision-recall curves for the two supervised models under the correct protocol; the dashed line is the prevalence (random) baseline.

Validation and robustness. To confirm the headline is not a lucky split, I cross-validate the two supervised pipelines with 5-fold stratified CV on the training data (SMOTE stays inside the pipeline, so folds do not leak). The means track the held-out numbers with small spread: random forest $F1\ 0.851 \pm 0.022$ / $PR-AUC\ 0.840 \pm 0.030$, logistic regression $F1\ 0.100 \pm 0.006$ / $PR-AUC\ 0.752 \pm 0.040$. The two anomaly baselines are instructive as a pair: at their default cutoffs they behave very differently (the isolation forest flags 191 rows, the Mahalanobis rule flags ~5,600), but by ranking (PR-AUC) the simple Mahalanobis model (0.417) actually beats the isolation forest (0.128) — a reminder that a fancier model is not automatically better. Both are well behind the supervised forest once labels are available.

Error analysis (random forest). The 37 missed frauds are dominated by tiny amounts (median 3.79 vs 30.39 for caught frauds), but include one transaction of 1,809.68, the single most expensive mistake in the test set. Misses are not concentrated in the two night-time fraud-rate spikes the EDA found: only 19% of misses fall in that window, against 30% of caught frauds. The “easy” night fraud gets caught; the hard cases hide in daytime traffic and sit closest to normal exactly on the features (V17, V10, V14) that carry the fraud signal. Because a missed fraud costs the transaction amount plus a chargeback while a false alarm costs minutes of review, I would lower the decision threshold. Turning that into money (notebook Section 7): if a false alarm costs a fixed review effort, the total-cost-minimising cutoff sits at about 0.18, not the default 0.5, and running at 0.5 costs about 1.57× the optimum.



Expected total cost (missed-fraud loss plus review effort) as a function of the random forest's decision threshold; the minimum sits well below the default 0.5.

6. Conclusions

Findings. (1) The tutorial's near-perfect result is an artifact of SMOTE-before-split; the same inflation reproduces with unrelated model families. (2) Accuracy and ROC-AUC actively mislead at 0.17% prevalence; PR-AUC, MCC and F2 tell the operational truth. (3) Honest performance on this dataset is useful but unglamorous: F1 around 0.83 for a class-weighted random forest, confirmed by cross-validation. (4) Two unsupervised anomaly detectors trained with no fraud labels still find a meaningful share of fraud (isolation forest 39/142; a Mahalanobis baseline ranks even better by PR-AUC), supporting the course's abnormality-detection framing that some fraud simply is an outlier.

Strengths and weaknesses of the solutions. The tutorial's approach has one real strength — it uses SMOTE, a legitimate imbalance tool, on a genuine public benchmark — but its fatal weakness is resampling before the split, which invalidates every headline number. My corrected pipeline's strengths are the leakage-free protocol, imbalance-aware headline metrics, a label-free anomaly baseline, and full reproducibility; its weaknesses are an F1 ceiling around 0.83, no probability calibration or hyperparameter tuning, and a single primary split (mitigated but not eliminated by the cross-validation).

Lessons. Resampling belongs inside the cross-validation / pipeline boundary, never before the split. Metric choice is not a formality; at this prevalence it decides whether a result means anything. And of the boastful repositories screened, some had correct code under inflated prose while the one critiqued here had the leak; only reading the code told them apart.

Future improvements. Cost-weighted threshold selection with real (not assumed) fraud-loss and review-cost figures — the notebook sketches this with an assumed cost; probability calibration; a temporal split (train on day 1, test on day 2) to test drift; and, given raw data access, the velocity and merchant features the anonymized set forbids.

7. Executive Summary

A public GitHub fraud-detection tutorial (sergi-s/Credit-Card-fraud-detection) claims its SMOTE-balanced neural network can “detect 100% of all fraudulent transactions in the unseen test set” (F1 0.998). This project reproduced the tutorial’s methodology on the same public ULB dataset (284,807 transactions, 0.173% fraud) and shows the claim is an artifact of data leakage: the tutorial applies SMOTE oversampling to the entire dataset before splitting into train and test, so the test set is contaminated with synthetic near-copies of training data, and its 50/50 balance makes precision high by construction.

Reproducing that flawed protocol with ordinary scikit-learn models yields the same near-perfect scores (logistic regression F1 0.979; gradient boosting F1 0.9995), confirming the numbers come from the protocol, not the model. Under a leakage-free protocol, with resampling confined to training folds inside an imblearn pipeline, the same logistic regression falls to F1 0.097, and the best honest model, a class-weighted random forest, reaches F1 0.830 / MCC 0.836 / PR-AUC 0.807 (cross-validated F1 0.851 ± 0.022), catching 105 of 142 test frauds with 6 false alarms in 85,118 transactions. Two label-free anomaly baselines (isolation forest and Mahalanobis distance) recover a smaller share of fraud, confirming that some fraud is detectable purely as an outlier but that supervised models win when labels exist. The tutorial’s own README corroborates the diagnosis: its reported F1 collapses from 0.998 to 0.570 when its model is scored on the original data, and the repository later gained “no leakage” notebooks.

The critique generalizes beyond one repository. At 0.17% prevalence, accuracy is meaningless (99.83% by predicting “no fraud” always) and ROC-AUC hides operational failure (two of the models here share $\text{ROC-AUC} \approx 0.94$ while their PR-AUCs differ six-fold). Error analysis shows the misses skew toward tiny amounts hiding in daytime traffic, and a cost model puts the deployable threshold near 0.18 rather than 0.5. Published analyses of this dataset’s literature (arXiv:2506.02703, arXiv:2412.07437) find these mistakes are widespread. The tutorial’s claims are not supported by its evidence; the corrected numbers are lower, and honest.

8. Summing It Up

This project critically evaluated the `sergi-s/Credit-Card-fraud-detection` tutorial, which trains fraud classifiers on the ULB `creditcard.csv` dataset (284,807 transactions, 0.173% fraud) — a canonical extreme-imbalance cybersecurity problem — using SMOTE oversampling and a Keras neural network. Did the reproduction succeed? Yes, in both directions. I reproduced the tutorial’s near-perfect numbers by copying its flawed protocol, and I reproduced the methodology correctly and obtained materially lower, credible performance. The most important insight is that a single misplaced preprocessing step — resampling before the split — manufactures the entire “100%” result, and the author’s own published tables (F1 collapsing 0.998 $\rightarrow$ 0.570) already contain the disproof. The author’s headline claims are therefore **not supported** by the evidence.

Would I recommend this approach for similar problems? Not as written. The corrected version of the same stack (split first; resampling inside pipelines; PR-AUC, MCC and F-beta as headline metrics; an anomaly model that needs no fraud labels as a baseline) is a reasonable template for rare-event detection problems in cybersecurity. What the tutorial actually demonstrates is different from what it claims: it is a case study in how the number a README advertises is worth exactly as much as the protocol behind it.

References

- ULB Machine Learning Group, “Credit Card Fraud Detection” dataset, Kaggle: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
- Tutorial under critique: <https://github.com/sergi-s/Credit-Card-fraud-detection> (accessed 7 July 2026)
- “Data Leakage and Deceptive Performance: A Critical Examination of Credit Card Fraud Detection Methodologies”, arXiv:2506.02703
- “Impact of Sampling Techniques and Data Leakage on XGBoost Performance in Credit Card Fraud Detection”, arXiv:2412.07437
- Lemaitre, Nogueira, Aridas, “Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning”, JMLR 18(17), 2017.